

ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

Trần Nguyễn Khải Luân
Nguyễn Bảo Duy
Trương Quốc Cường
Nguyễn Thanh Tiến

Nghiên cứu về khả năng học chính xác thuật toán của mạng nơ-ron

*Bài báo: Learning to Add, Multiply, and Execute Algorithmic Instructions
Exactly with Neural Networks*

Môn học: Nhập môn Học máy

TP. Hồ Chí Minh, Năm 2025

ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

Trần Nguyễn Khải Luân
Nguyễn Bảo Duy
Trương Quốc Cường
Nguyễn Thanh Tiến

Nghiên cứu về khả năng học chính xác thuật toán của mạng nơ-ron

*Bài báo: Learning to Add, Multiply, and Execute Algorithmic Instructions
Exactly with Neural Networks*

Môn học: Nhập môn Học máy

Giảng viên hướng dẫn: ThS. Lê Nhật Nam

TP. Hồ Chí Minh, Năm 2025

THÔNG TIN NHÓM

- **Nhóm:** 11
- **Nhóm trưởng:** Trương Quốc Cường - 23127333
- **Danh sách thành viên:**
 1. Trần Nguyễn Khải Luân - 23127006
 2. Nguyễn Bảo Duy - 23127179
 3. Nguyễn Thanh Tiến - 23127539

Mục lục

1	TỔNG QUAN	1
1.1	Giới thiệu	1
1.2	Các nghiên cứu liên quan	1
1.3	Động lực nghiên cứu	2
1.4	Câu hỏi nghiên cứu	2
1.5	Đóng góp của báo cáo	3
2	CƠ SỞ LÝ THUYẾT	4
2.1	Ký hiệu và các khái niệm cơ bản	4
2.2	Neural Tangent Kernel (NTK)	4
2.3	Giới hạn chiều rộng vô hạn và Động lực học	5
2.3.1	Chế độ Lazy Training	5
2.3.2	Sự ổn định của Kernel (Kernel đóng băng)	6
2.3.3	Phương trình vi phân cho đầu ra mạng	6
2.3.4	Tương đương với Kernel Regression	6
2.4	Kiến trúc mạng nơ-ron hai lớp	7
2.4.1	Cấu trúc mạng	7
2.4.2	NTK Parametrization	7
2.5	Điều kiện về khả năng học chính xác	8
2.5.1	Assumption 5.1: Kiểm soát tương quan không mong muốn	8
2.5.2	Vai trò của hàm làm tròn	8
3	PHƯƠNG PHÁP ĐỀ XUẤT	10
3.1	Kỹ thuật Template Matching	10
3.1.1	Nguyên lý phân rã thuật toán	10
3.1.2	Công thức hàm tổng hợp	10
3.1.3	Cơ chế thực thi lặp	11
3.2	Cấu trúc dữ liệu và Trục giao hóa	11

3.2.1	Mô hình “Bag of Rules”	11
3.2.2	Mã hóa mẫu thành vector huấn luyện	11
3.2.3	Trực giao hóa dữ liệu huấn luyện	12
3.3	Định lý về bộ dự báo NTK	12
3.3.1	Phát biểu Định lý 5.1	12
3.3.2	Ý nghĩa và ứng dụng	12
3.4	Kiểm soát tương quan không mong muốn	13
3.4.1	Chi tiết Assumption 5.1	13
3.4.2	Điều kiện đủ để học chính xác	13
3.4.3	Xác minh cho các thuật toán cụ thể	13
3.5	Triển khai thuật toán cụ thể	14
3.5.1	Phép hoán vị	14
3.5.2	Phép cộng	14
3.5.3	Phép nhân	15
3.5.4	Chỉ thị SBN (Subtract and Branch if Negative)	15
3.6	Độ phức tạp tập hợp	15
3.6.1	Tại sao cần Ensemble	16
3.6.2	Công thức tính kích thước Ensemble	16
3.6.3	Độ phức tạp theo số bit	16
3.6.4	So sánh với độ phức tạp mẫu	17
4	THỰC NGHIỆM VÀ PHÂN TÍCH	18
4.1	Thiết lập thực nghiệm	18
4.2	Xác minh lý thuyết	18
4.2.1	Kết quả trên phép cộng và nhân	18
4.2.2	So sánh phương sai và kỳ vọng	18
4.3	Thực nghiệm huấn luyện	18
4.3.1	Bài toán Hoán vị	18
4.3.2	Độ chính xác theo kích thước Ensemble	18
5	KẾT LUẬN	19
A	Chứng minh toán học bổ trợ	21
B	Mã nguồn cài đặt mô phỏng	22

Danh sách bảng

Danh sách hình vẽ

Chương 1

TỔNG QUAN

1.1 Giới thiệu

Mạng nơ-ron đã chứng minh khả năng xấp xỉ vượt trội đối với các hàm liên tục và trơn, đặc biệt trong các bài toán nhận dạng mẫu, xử lý ngôn ngữ tự nhiên và thị giác máy tính. Tuy nhiên, khi đối mặt với các phép toán rời rạc như phép cộng nhị phân, phép nhân số học hay thực thi các chỉ thị thuật toán, mạng nơ-ron truyền thống thường gặp khó khăn đáng kể. Sự đối lập này phản ánh thách thức cơ bản: trong khi mạng nơ-ron được tối ưu hóa cho việc xấp xỉ hàm số trong không gian liên tục, các thuật toán thực tế yêu cầu sự chính xác tuyệt đối ở cấp độ bit.

Nghiên cứu đã đề xuất một hướng tiếp cận đột phá để giải quyết vấn đề này thông qua khung lý thuyết Neural Tangent Kernel (NTK). Bằng cách phân tích mạng nơ-ron hai lớp trong giới hạn chiều rộng vô hạn, các tác giả chứng minh rằng mạng nơ-ron có thể được huấn luyện để thực thi chính xác các chỉ thị thuật toán được mã hóa ở dạng nhị phân. Đây là kết quả có ý nghĩa lý thuyết quan trọng, mở ra khả năng sử dụng mạng nơ-ron như một công cụ tính toán phổ quát thay vì chỉ giới hạn ở vai trò xấp xỉ hàm số.

Mục tiêu cốt lõi của nghiên cứu là chứng minh một cách chặt chẽ về mặt toán học rằng mạng nơ-ron, khi được huấn luyện thông qua gradient descent với cơ chế template matching, có khả năng học và thực thi chính xác các chỉ thị thuật toán mà không cần xấp xỉ.

1.2 Các nghiên cứu liên quan

Khả năng tính toán của mạng nơ-ron đã được nghiên cứu rộng rãi trong nhiều thập kỷ qua. Các nghiên cứu tiên phong về Recurrent Neural Networks (RNN) đã chứng minh rằng RNN có thể mô phỏng máy Turing (Turing-complete) về mặt lý thuyết, nghĩa là chúng có khả năng biểu diễn bất kỳ hàm tính toán được (computable function) nào. Gần đây hơn, kiến trúc Transformer cũng được chứng minh có khả năng biểu diễn tương đương, củng cố thêm quan điểm về tính phổ quát của mạng nơ-ron.

Tuy nhiên, điều quan trọng cần phân biệt là tính khả thi về mặt biểu diễn (expressive power) hoàn toàn khác với khả năng học được (learnability) thông qua gradient descent. Một mạng nơ-ron có thể có khả năng biểu diễn một hàm phức tạp, nhưng điều đó không đảm bảo rằng thuật toán tối ưu hóa như gradient descent có thể tìm được tập tham số phù hợp để thực hiện hàm đó từ dữ liệu huấn luyện hữu hạn. Đây chính là khoảng cách then chốt mà các nghiên cứu trước đó chưa giải quyết triệt để.

Nhiều kiến trúc chuyên biệt đã được đề xuất để thu hẹp khoảng cách này. Neural GPU mở rộng khả năng của mạng tích chập để xử lý các phép toán số học song song. Neural Arithmetic Logic Unit (NALU) được thiết kế đặc biệt để học các phép toán số học như cộng, trừ, nhân và chia. Mặc dù các kiến trúc này đạt được kết quả thực nghiệm tốt trong một số trường hợp, chúng thiếu nền tảng lý thuyết vững chắc về việc tại sao và khi nào quá trình học hội tụ đến giải pháp chính xác.

Nghiên cứu đã cho thấy sự khác biệt cơ bản ở chỗ chứng minh lý thuyết chặt chẽ dựa trên khung NTK, đảm bảo rằng mạng nơ-ron không chỉ có khả năng biểu diễn mà thực sự có thể học được các phép toán chính xác thông qua gradient descent với lượng dữ liệu huấn luyện có thể kiểm soát được.

1.3 Động lực nghiên cứu

Một trong những thách thức cốt lõi khi huấn luyện mạng nơ-ron thực hiện các phép toán rời rạc là sự khó khăn trong việc hội tụ gradient descent đối với các hàm không liên tục. Gradient descent hoạt động dựa trên việc tính toán đạo hàm của hàm mất mát theo các tham số mạng, nhưng đối với các hàm bước nhảy hay các phép toán nhị phân, đạo hàm hầu như bằng không hoặc không xác định tại hầu hết các điểm. Điều này dẫn đến hiện tượng gradient biến mất (vanishing gradient) hoặc gradient không cung cấp thông tin hữu ích cho việc cập nhật tham số.

Các bộ dữ liệu tiêu chuẩn cho bài toán số học thường không được thiết kế để giải quyết vấn đề này một cách có hệ thống. Hệ quả là mạng nơ-ron được huấn luyện trên các bộ dữ liệu như vậy có thể đạt độ chính xác cao trên tập huấn luyện nhưng thất bại khi khái quát hóa (generalization) sang các ví dụ mới, đặc biệt khi kích thước đầu vào tăng lên.

Động lực chính của nghiên cứu này xuất phát từ nhu cầu tìm kiếm một cơ chế huấn luyện có thể đảm bảo mạng nơ-ron đạt được độ chính xác tuyệt đối (exact execution) thay vì chỉ xấp xỉ. Thay vì cố gắng xấp xỉ một hàm rời rạc bằng một hàm liên tục, nghiên cứu này đề xuất mã hóa dữ liệu và thuật toán theo cách cho phép mạng nơ-ron nhận dạng và áp dụng các quy tắc cấp độ bit một cách chính xác thông qua cơ chế template matching. Cách tiếp cận này biến bài toán từ xấp xỉ hàm thành bài toán nhận dạng mẫu, nơi mạng nơ-ron có thể phát huy thế mạnh vốn có.

1.4 Câu hỏi nghiên cứu

Câu hỏi nghiên cứu trọng tâm mà tác giả đặt ra là:

Liệu mạng nơ-ron có thể học thực thi chính xác các chỉ thị thuật toán thông qua cơ chế template matching, và nếu có thể, lượng dữ liệu huấn luyện tối thiểu cần thiết là bao nhiêu?

Câu hỏi này bao hàm nhiều khía cạnh quan trọng:

- **Tính khả thi về mặt học:** Liệu gradient descent có thể hội tụ đến một giải pháp thực hiện chính xác thuật toán hay không?
- **Hiệu quả dữ liệu:** Số lượng ví dụ huấn luyện cần thiết có thể được kiểm soát ở mức nào? Liệu có thể đạt được độ phức tạp mẫu logarithmic thay vì tuyến tính hay đa thức theo kích thước đầu vào?

- **Tính phổ quát:** Kết quả có thể mở rộng cho lớp các chỉ thị thuật toán nào? Liệu có thể đạt được tính Turing-complete không?

1.5 Đóng góp của báo cáo

Nghiên cứu của đã mang lại 5 đóng góp chính về mặt lý thuyết và phương pháp luận:

Thứ nhất, các tác giả đề xuất phương pháp biểu diễn thuật toán dưới dạng template (Algorithmic Template Representation). Trong phương pháp này, mỗi quy tắc cấp độ bit của thuật toán được mã hóa thành một vector mẫu riêng biệt. Cách tiếp cận này cho phép cô lập các quy tắc logic và tạo ra cấu trúc dữ liệu phù hợp cho việc áp dụng cơ chế template matching, nơi mạng nơ-ron học cách nhận dạng và áp dụng từng quy tắc một cách chính xác.

Thứ hai, nghiên cứu cung cấp chứng minh lý thuyết chặt chẽ dựa trên khung Neural Tangent Kernel (NTK) về khả năng học chính xác các phép toán cơ bản. Cụ thể, các tác giả chứng minh rằng mạng nơ-ron hai lớp trong giới hạn infinite-width có thể học chính xác:

- Phép hoán vị (permutation) trên các bit
- Phép cộng nhị phân (binary addition)
- Phép nhân nhị phân (binary multiplication)
- Chỉ thị Subtract and Branch if Negative (SBN)

Thứ ba, nghiên cứu chứng minh tính hiệu quả về dữ liệu của phương pháp đề xuất. Thay vì yêu cầu số lượng ví dụ huấn luyện tỷ lệ tuyến tính hoặc đa thức với kích thước đầu vào, phương pháp này chỉ cần số lượng ví dụ ở quy mô logarithmic. Điều này có ý nghĩa thực tiễn quan trọng, cho phép huấn luyện hiệu quả ngay cả với các bài toán có kích thước đầu vào lớn.

Thứ tư, nghiên cứu khẳng định tính phổ quát của khung lý thuyết đề xuất. Vì chỉ thị SBN đã được chứng minh là Turing-complete (tức là bất kỳ hàm tính toán được nào cũng có thể được biểu diễn bằng một chuỗi các chỉ thị SBN), việc chứng minh mạng nơ-ron có thể học chính xác SBN đồng nghĩa với việc mạng nơ-ron có thể, về nguyên tắc, học thực thi bất kỳ thuật toán nào có thể được mô tả bằng một chuỗi hữu hạn các chỉ thị.

Thứ năm, báo cáo này tổng hợp và trình bày lại các kết quả từ nghiên cứu gốc dưới góc nhìn giáo dục, giúp người đọc tiếp cận các khái niệm phức tạp như NTK, infinite-width limit, và template matching một cách có hệ thống và dễ hiểu hơn.

Các đóng góp này không chỉ có giá trị về mặt lý thuyết thuần túy mà còn mở ra hướng phát triển mới cho việc thiết kế và huấn luyện mạng nơ-ron trong các ứng dụng yêu cầu độ chính xác tuyệt đối, như xác minh phần cứng, mã hóa và giải mã, và các hệ thống nhúng quan trọng.

Chương 2

CƠ SỞ LÝ THUYẾT

2.1 Ký hiệu và các khái niệm cơ bản

Trước khi đi vào phân tích chi tiết khung lý thuyết Neural Tangent Kernel (NTK), chúng ta cần thiết lập các ký hiệu toán học cơ bản được sử dụng xuyên suốt báo cáo này.

Vector và ma trận: Một vector cột n chiều được ký hiệu là $\mathbf{x} \in \mathbb{R}^n$, trong đó phần tử thứ i được ký hiệu là x_i hoặc $[\mathbf{x}]_i$. Ma trận đơn vị kích thước $n \times n$ được ký hiệu là I_n . Tích vô hướng giữa hai vector $\mathbf{x}$ và $\mathbf{y}$ được ký hiệu là $\langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x}^\top \mathbf{y} = \sum_{i=1}^n x_i y_i$.

Chuẩn Euclidean: Chuẩn Euclidean (hay chuẩn L^2) của một vector $\mathbf{x}$ được định nghĩa là:

$$\|\mathbf{x}\| = \sqrt{\sum_{i=1}^n x_i^2} = \sqrt{\langle \mathbf{x}, \mathbf{x} \rangle} \quad (2.1)$$

Tập hợp chỉ số: Ký hiệu $[n]$ biểu thị tập hợp các số nguyên dương từ 1 đến n :

$$[n] = \{1, 2, \dots, n\} \quad (2.2)$$

Bài toán hồi quy và hàm loss: Xét một mạng nơ-ron với tham số θ thực hiện ánh xạ đầu vào $\mathbf{x}$ sang đầu ra $F(\mathbf{x}; \theta)$. Cho tập dữ liệu huấn luyện $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N$ với N mẫu, mục tiêu huấn luyện là tối thiểu hóa hàm loss Mean Squared Error (MSE):

$$\mathcal{L}(\theta) = \frac{1}{2} \|F(\mathbf{x}; \theta) - \mathbf{y}\|^2 = \frac{1}{2} \sum_{i=1}^N \|F(\mathbf{x}_i; \theta) - \mathbf{y}_i\|^2 \quad (2.3)$$

Trong đó $\mathbf{y}$ biểu thị vector chứa tất cả các nhãn đầu ra mục tiêu. Hàm loss MSE có đạo hàm đơn giản và là lựa chọn tiêu chuẩn cho các bài toán hồi quy, đồng thời cũng là nền tảng cho việc phân tích lý thuyết trong khung NTK.

2.2 Neural Tangent Kernel (NTK)

Neural Tangent Kernel (NTK) là một công cụ lý thuyết mạnh mẽ cho phép phân tích động lực học huấn luyện của mạng nơ-ron. Ý tưởng cốt lõi là mô tả sự thay đổi của đầu ra mạng theo thời gian huấn luyện thông qua một kernel cố định.

Ma trận Jacobian: Xét mạng nơ-ron với đầu ra $F(\mathbf{x}; \theta) \in \mathbb{R}^k$ phụ thuộc vào vector tham số $\theta \in \mathbb{R}^p$. Ma trận Jacobian của đầu ra theo tham số được định nghĩa là:

$$J(\theta) = \frac{\partial F(\mathbf{x}; \theta)}{\partial \theta} \in \mathbb{R}^{Nk \times p} \quad (2.4)$$

trong đó N là số lượng mẫu dữ liệu, k là chiều của đầu ra, và p là số lượng tham số. Mỗi hàng của ma trận Jacobian biểu thị gradient của một thành phần đầu ra theo toàn bộ tham số.

Định nghĩa NTK thực nghiệm: Neural Tangent Kernel thực nghiệm (empirical NTK) được định nghĩa thông qua tích của ma trận Jacobian với chuyển vị của nó:

$$K(\theta) = J(\theta)J(\theta)^\top \in \mathbb{R}^{Nk \times Nk} \quad (2.5)$$

Phần tử (i, j) của ma trận kernel này đo lường mức độ tương quan giữa gradient của đầu ra thứ i và đầu ra thứ j trong không gian tham số. Cụ thể hơn, với hai điểm dữ liệu $\mathbf{x}$ và $\mathbf{x}'$, giá trị kernel được tính như sau:

$$K(\mathbf{x}, \mathbf{x}'; \theta) = \left\langle \frac{\partial F(\mathbf{x}; \theta)}{\partial \theta}, \frac{\partial F(\mathbf{x}'; \theta)}{\partial \theta} \right\rangle \quad (2.6)$$

Ý nghĩa vật lý của NTK: NTK cho phép hiểu rõ cách mạng nơ-ron “học” từ dữ liệu. Khi huấn luyện bằng gradient descent, sự thay đổi của đầu ra mạng tại một điểm $\mathbf{x}$ không chỉ phụ thuộc vào gradient tại điểm đó mà còn phụ thuộc vào gradient tại tất cả các điểm dữ liệu khác thông qua kernel. Đây chính là cơ chế “học chuyển giao” (transfer learning) nội tại trong mạng nơ-ron: thông tin học được từ một mẫu có thể ảnh hưởng đến dự đoán tại các mẫu khác.

2.3 Giới hạn chiều rộng vô hạn và Động lực học

Một trong những đóng góp quan trọng nhất của lý thuyết NTK là khám phá hành vi của mạng nơ-ron khi chiều rộng lớp ẩn tiến tới vô hạn. Trong giới hạn này, việc huấn luyện mạng nơ-ron trở nên đơn giản hóa đáng kể và có thể được phân tích chính xác.

2.3.1 Chế độ Lazy Training

Khi chiều rộng lớp ẩn $m \rightarrow \infty$, mạng nơ-ron hoạt động trong chế độ được gọi là *lazy training*. Trong chế độ này, các tham số của mạng thay đổi cực nhỏ quanh điểm khởi tạo trong suốt quá trình huấn luyện:

$$\|\theta(t) - \theta(0)\| \approx O\left(\frac{1}{\sqrt{m}}\right) \rightarrow 0 \quad \text{khi } m \rightarrow \infty \quad (2.7)$$

Điều này có vẻ nghịch lý: làm sao mạng có thể học được nếu tham số hầu như không thay đổi? Câu trả lời nằm ở việc với số lượng tham số cực lớn, ngay cả những thay đổi nhỏ trong mỗi tham số cũng đủ để tạo ra thay đổi đáng kể trong đầu ra. Đây chính là sức mạnh của việc có nhiều chiều trong không gian tham số.

2.3.2 Sự ổn định của Kernel (Kernel đóng băng)

Một hệ quả quan trọng của lazy training là NTK trở nên ổn định (“đóng băng”) trong suốt quá trình huấn luyện. Cụ thể, khi $m \rightarrow \infty$:

$$K_m(\theta(t)) \rightarrow \Theta(\mathbf{x}, \mathbf{x}') \quad \text{với mọi } t \geq 0 \quad (2.8)$$

trong đó $\Theta(\mathbf{x}, \mathbf{x}')$ là *infinite-width NTK*, một kernel xác định và cố định chỉ phụ thuộc vào cặp đầu vào $(\mathbf{x}, \mathbf{x}')$ và kiến trúc mạng, không phụ thuộc vào quá trình huấn luyện. Sự ổn định này là chìa khóa cho phép phân tích chính xác động lực học huấn luyện.

2.3.3 Phương trình vi phân cho đầu ra mạng

Xét việc huấn luyện mạng bằng gradient flow (gradient descent với learning rate vô cùng nhỏ). Động lực học của tham số được mô tả bởi:

$$\frac{d\theta}{dt} = -\nabla_{\theta} \mathcal{L}(\theta) = -J(\theta)^{\top} (F(\mathbf{x}; \theta) - \mathbf{y}) \quad (2.9)$$

Từ đây, có thể suy ra phương trình vi phân cho đầu ra mạng. Đặt $f(t) = F(\mathbf{x}; \theta(t))$ là vector đầu ra tại thời điểm t , ta có:

$$\frac{df}{dt} = J(\theta) \frac{d\theta}{dt} = -J(\theta)J(\theta)^{\top} (f(t) - \mathbf{y}) = -K(\theta(t))(f(t) - \mathbf{y}) \quad (2.10)$$

Trong giới hạn infinite-width, với $K(\theta(t)) \approx \Theta$ cố định, phương trình trên trở thành phương trình vi phân tuyến tính:

$$\frac{df}{dt} = -\Theta(f(t) - \mathbf{y}) \quad (2.11)$$

Nghiệm của phương trình này có dạng giải tích tường minh:

$$f(t) = \mathbf{y} + e^{-\Theta t} (f(0) - \mathbf{y}) \quad (2.12)$$

trong đó $e^{-\Theta t}$ là exponential của ma trận. Khi $t \rightarrow \infty$, nếu Θ là ma trận xác định dương, ta có $f(t) \rightarrow \mathbf{y}$, nghĩa là mạng học được chính xác nhãn đầu ra.

2.3.4 Tương đương với Kernel Regression

Kết quả quan trọng nhất của phân tích trên là việc huấn luyện mạng nơ-ron trong giới hạn infinite-width hoàn toàn tương đương với bài toán *kernel regression* cổ điển. Cho một điểm test mới $\mathbf{x}^*$, dự đoán của mạng sau khi huấn luyện hội tụ là:

$$f_{\infty}(\mathbf{x}^*) = \Theta(\mathbf{x}^*, \mathcal{X}) \Theta(\mathcal{X}, \mathcal{X})^{-1} \mathbf{y} \quad (2.13)$$

trong đó:

- $\mathcal{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ là tập các điểm huấn luyện
- $\Theta(\mathbf{x}^*, \mathcal{X}) \in \mathbb{R}^{1 \times N}$ là vector kernel giữa điểm test và các điểm huấn luyện
- $\Theta(\mathcal{X}, \mathcal{X}) \in \mathbb{R}^{N \times N}$ là ma trận kernel (Gram matrix) trên tập huấn luyện

Công thức này chính là nghiệm của bài toán kernel regression tiêu chuẩn. Điều này cho thấy mạng nơ-ron sâu, trong giới hạn chiều rộng vô hạn, hoạt động như một phương pháp kernel cổ điển với một kernel đặc biệt được xác định bởi kiến trúc mạng.

2.4 Kiến trúc mạng nơ-ron hai lớp

Nghiên cứu của Back de Luca và cộng sự tập trung vào mạng nơ-ron hai lớp với kiến trúc đơn giản nhưng đủ mạnh để phân tích lý thuyết.

2.4.1 Cấu trúc mạng

Mạng nơ-ron hai lớp được sử dụng có cấu trúc như sau:

- **Lớp đầu vào:** Nhận vector $\mathbf{x} \in \mathbb{R}^d$
- **Lớp ẩn:** Gồm m nơ-ron với hàm kích hoạt ReLU, không sử dụng bias
- **Lớp đầu ra:** Ánh xạ tuyến tính từ m nơ-ron ẩn sang k đầu ra

Công thức toán học của mạng được viết là:

$$F(\mathbf{x}) = W^2 \cdot \text{ReLU}(W^1 \mathbf{x}) \quad (2.14)$$

trong đó:

- $W^1 \in \mathbb{R}^{m \times d}$ là ma trận trọng số lớp thứ nhất
- $W^2 \in \mathbb{R}^{k \times m}$ là ma trận trọng số lớp thứ hai
- $\text{ReLU}(z) = \max(0, z)$ là hàm kích hoạt Rectified Linear Unit, áp dụng element-wise

Việc không sử dụng bias đơn giản hóa phân tích lý thuyết mà không làm mất tính tổng quát về mặt biểu diễn, vì bias có thể được mô phỏng bằng cách thêm một chiều hằng số vào đầu vào.

2.4.2 NTK Parametrization

Để đảm bảo tính hội tụ trong giới hạn infinite-width, các trọng số được khởi tạo theo chuẩn *NTK parametrization*. Cụ thể:

- Các phần tử của W^1 được khởi tạo i.i.d. từ phân phối $\mathcal{N}(0, 1)$
- Các phần tử của W^2 được khởi tạo i.i.d. từ phân phối $\mathcal{N}(0, 1/m)$

Hệ số $1/m$ trong phương sai của W^2 là then chốt. Nó đảm bảo rằng đầu ra của mạng có phương sai $O(1)$ khi $m \rightarrow \infty$:

$$\text{Var}[F(\mathbf{x})] = \text{Var} \left[\sum_{j=1}^m W_{ij}^2 \cdot \text{ReLU}(W_j^1 \mathbf{x}) \right] \approx m \cdot \frac{1}{m} \cdot O(1) = O(1) \quad (2.15)$$

Nếu không có hệ số này, đầu ra sẽ có phương sai tăng theo m , làm mất tính ổn định khi $m \rightarrow \infty$.

2.5 Điều kiện về khả năng học chính xác

Để mạng nơ-ron có thể học thực thi chính xác các chỉ thị thuật toán, cần có các điều kiện đảm bảo rằng thông tin tín hiệu không bị nhiễu làm suy giảm. Phần này trình bày các điều kiện then chốt được đề xuất trong nghiên cứu.

2.5.1 Assumption 5.1: Kiểm soát tương quan không mong muốn

Xét một bước tính toán của thuật toán với đầu vào được biểu diễn dưới dạng template. Đầu ra của bộ dự đoán NTK có thể được phân tích thành hai thành phần:

$$\hat{y} = w^1 \cdot (\text{tín hiệu chính xác}) + w^0 \cdot (\text{tổng các tương quan không mong muốn}) \quad (2.16)$$

trong đó:

- w^1 là *trọng số tín hiệu* (signal weight): Đóng góp từ các template khớp chính xác với quy tắc cần áp dụng
- w^0 là *trọng số nhiễu* (interference weight): Đóng góp từ các template không liên quan nhưng có tương quan khác không với đầu vào

Định nghĩa tương quan không mong muốn: Một tương quan được gọi là “không mong muốn” (unwanted correlation) khi một template trong tập huấn luyện có tích vô hướng khác không với đầu vào hiện tại, mặc dù template đó không đại diện cho quy tắc cần áp dụng. Gọi C là số lượng các tương quan không mong muốn như vậy.

Điều kiện đủ để học chính xác: Để đầu ra $\hat{y}$ có dấu đúng (từ đó có thể làm tròn về giá trị chính xác), cần đảm bảo rằng thành phần tín hiệu lấn át thành phần nhiễu:

$$|w^1| > C \cdot |w^0| \quad (2.17)$$

hay tương đương:

$$C < \frac{|w^1|}{|w^0|} \quad (2.18)$$

Điều kiện này có ý nghĩa trực quan: số lượng tương quan không mong muốn phải đủ nhỏ so với tỷ lệ giữa trọng số tín hiệu và trọng số nhiễu. Trong thiết kế template của nghiên cứu này, các tác giả đã chứng minh rằng với cách mã hóa phù hợp, tỷ lệ $|w^1|/|w^0|$ tăng theo kích thước bài toán, trong khi C chỉ tăng theo logarithm, đảm bảo điều kiện trên được thỏa mãn.

2.5.2 Vai trò của hàm làm tròn

Một thành phần quan trọng trong khung lý thuyết là việc áp dụng hàm làm tròn (rounding) sau mỗi bước dự đoán. Cụ thể:

$$y_{\text{output}} = \text{round}(\hat{y}) = \begin{cases} +1 & \text{nếu } \hat{y} > 0 \\ -1 & \text{nếu } \hat{y} < 0 \end{cases} \quad (2.19)$$

Hàm làm tròn đóng vai trò then chốt trong việc:

1. **Loại bỏ nhiễu:** Miễn là đầu ra thô $\hat{y}$ có dấu đúng, giá trị chính xác được khôi phục hoàn toàn sau khi làm tròn. Điều này cho phép chấp nhận một lượng nhiễu nhất định mà không ảnh hưởng đến kết quả cuối cùng.
2. **Ngăn chặn tích lũy sai số:** Trong các thuật toán đa bước, nếu không có làm tròn, sai số nhỏ ở các bước trước sẽ tích lũy và phóng đại qua các bước sau. Làm tròn tại mỗi bước đảm bảo rằng đầu ra luôn chính xác tuyệt đối, ngăn chặn hiện tượng này.
3. **Chuyển đổi xấp xỉ thành chính xác:** Trong khi bộ dự đoán NTK về bản chất là một bộ xấp xỉ liên tục, việc kết hợp với làm tròn biến nó thành một hệ thống thực thi chính xác các phép toán rời rạc.

Kết hợp Assumption 5.1 và cơ chế làm tròn, nghiên cứu chứng minh rằng mạng nơ-ron trong giới hạn infinite-width có khả năng học và thực thi chính xác các chỉ thị thuật toán, mở đường cho các kết quả về phép cộng, phép nhân, và chỉ thị SBN được trình bày trong các chương tiếp theo.

Chương 3

PHƯƠNG PHÁP ĐỀ XUẤT

3.1 Kỹ thuật Template Matching

Ý tưởng cốt lõi của phương pháp đề xuất là biến đổi bài toán thực thi thuật toán thành bài toán nhận dạng mẫu (pattern recognition), nơi mạng nơ-ron có thể phát huy thế mạnh vốn có. Thay vì cố gắng học trực tiếp một hàm rời rạc phức tạp, thuật toán được phân rã thành các quy tắc cục bộ đơn giản, mỗi quy tắc được mã hóa thành một “mẫu” (template).

3.1.1 Nguyên lý phân rã thuật toán

Xét một thuật toán thao tác trên dữ liệu nhị phân có l bit. Thay vì xem thuật toán như một hàm đơn nhất $f : \{0, 1\}^l \rightarrow \{0, 1\}^l$, phương pháp template matching phân rã nó thành tập hợp các hàm khớp mẫu cục bộ $\{f_i\}$, mỗi hàm thao tác trên một khối bit nhỏ (block bit).

Mỗi hàm cục bộ f_i được định nghĩa trên một tập con các bit đầu vào và xác định giá trị của một bit đầu ra cụ thể. Cấu trúc này được gọi là *block-partitioned*, trong đó:

- Đầu vào được chia thành các khối bit có kích thước cố định
- Mỗi quy tắc cục bộ chỉ phụ thuộc vào một số khối đầu vào giới hạn
- Đầu ra của mỗi quy tắc xác định một hoặc vài bit cụ thể

3.1.2 Công thức hàm tổng hợp

Cho $\hat{x}$ là vector trạng thái hiện tại (biểu diễn nhị phân mở rộng), hàm tổng hợp $f(\hat{x})$ được xây dựng từ các hàm cục bộ thông qua phép tuyển bitwise (bitwise disjunction):

$$f(\hat{x}) = \bigvee_i f_i(\hat{x}) \quad (3.1)$$

trong đó $\bigvee$ biểu thị phép OR logic theo từng bit. Mỗi hàm $f_i(\hat{x})$ trả về một vector có hầu hết các thành phần bằng 0, chỉ khác 0 tại vị trí bit mà quy tắc đó xác định. Phép tuyển kết hợp tất cả các đóng góp này thành vector đầu ra hoàn chỉnh.

3.1.3 Cơ chế thực thi lặp

Nhiều thuật toán yêu cầu nhiều bước tính toán tuần tự. Phương pháp đề xuất mô hình hóa điều này thông qua cơ chế thực thi lặp (iterative execution):

$$\hat{x}_{t+1} = f(\hat{x}_t) \quad (3.2)$$

trong đó $\hat{x}_t$ là trạng thái tại bước thời gian t . Quá trình lặp tiếp tục cho đến khi đạt được trạng thái cuối cùng (ví dụ: sau một số bước cố định hoặc khi đạt điều kiện dừng).

Điểm quan trọng là tại mỗi bước, đầu ra được làm tròn (rounding) về giá trị nhị phân chính xác trước khi đưa vào bước tiếp theo. Điều này ngăn chặn sự tích lũy sai số và đảm bảo tính chính xác xuyên suốt quá trình thực thi.

3.2 Cấu trúc dữ liệu và Trục giao hóa

Để áp dụng khung lý thuyết NTK một cách hiệu quả, dữ liệu huấn luyện cần được cấu trúc theo cách đặc biệt. Phần này mô tả chiến lược mã hóa và tầm quan trọng của việc trục giao hóa.

3.2.1 Mô hình “Bag of Rules”

Chiến lược mã hóa trung tâm là gom tất cả các quy tắc cục bộ của thuật toán vào một không gian vector chung, tạo thành mô hình “Bag of Rules”. Trong mô hình này:

- Mỗi quy tắc cục bộ được biểu diễn bởi một cặp (đầu vào mẫu, đầu ra mong đợi)
- Tất cả các cặp được gom thành tập dữ liệu huấn luyện duy nhất
- Mạng nơ-ron học cách ánh xạ từ mẫu đầu vào sang đầu ra tương ứng

Ưu điểm của cách tiếp cận này là tính module: mỗi quy tắc được học độc lập, và việc thêm quy tắc mới không yêu cầu thay đổi cấu trúc mạng.

3.2.2 Mã hóa mẫu thành vector huấn luyện

Mỗi mẫu (template) được mã hóa thành vector q_{ij} có cấu trúc *block-partitioned*. Cụ thể, với quy tắc thứ j của bit đầu ra thứ i :

$$q_{ij} = [\underbrace{0, \dots, 0}_{\text{khối 1}}, \underbrace{v_1, \dots, v_k}_{\text{khối liên quan}}, \underbrace{0, \dots, 0}_{\text{khối cuối}}] \quad (3.3)$$

trong đó các khối liên quan chứa giá trị nhị phân mã hóa điều kiện của quy tắc, còn các khối không liên quan được đặt bằng 0. Cấu trúc block-partitioned này đảm bảo rằng các quy tắc thao tác trên các khối khác nhau có vector biểu diễn trục giao với nhau.

3.2.3 Trục giao hóa dữ liệu huấn luyện

Việc trục giao hóa (orthogonalization) dữ liệu huấn luyện đóng vai trò then chốt trong phân tích lý thuyết. Khi các vector mẫu được thiết kế sao cho gần trục giao với nhau, ma trận NTK có cấu trúc đặc biệt thuận lợi.

Cụ thể, với dữ liệu được trục giao hóa tốt, ma trận Gram $\Theta(\mathcal{X}, \mathcal{X})$ tiệm cận dạng:

$$\Theta(\mathcal{X}, \mathcal{X}) \approx \alpha I_N + \beta \mathbf{1}\mathbf{1}^\top \quad (3.4)$$

trong đó:

- α là hệ số scale chính
- I_N là ma trận đơn vị kích thước $N \times N$
- $\beta \mathbf{1}\mathbf{1}^\top$ là thành phần nhiễu rank-1

Cấu trúc “scaled identity + rank-1 noise” này cho phép nghịch đảo ma trận một cách giải tích và phân tích chính xác hành vi của bộ dự đoán NTK.

3.3 Định lý về bộ dự báo NTK

Phần này trình bày kết quả lý thuyết trung tâm về hành vi của bộ dự đoán trong chế độ NTK.

3.3.1 Phát biểu Định lý 5.1

Định lý 5.1 (NTK Predictor Behavior): Xét mạng nơ-ron hai lớp trong giới hạn infinite-width được huấn luyện trên tập dữ liệu template. Cho $\hat{x}$ là một đầu vào test và $\mu(\hat{x})$ là kỳ vọng của phân phối giới hạn của đầu ra mạng. Khi đó, với mỗi bit đầu ra thứ i :

$$\begin{cases} \mu(\hat{x})_i > 0 & \text{nếu bit đầu ra đúng là } +1 \\ \mu(\hat{x})_i \leq 0 & \text{nếu bit đầu ra đúng là } -1 \end{cases} \quad (3.5)$$

Định lý này khẳng định rằng bộ dự đoán NTK bảo toàn thông tin về dấu (sign) của bit đầu ra. Mặc dù giá trị số cụ thể của $\mu(\hat{x})_i$ có thể không chính xác bằng ± 1 , dấu của nó luôn đúng.

3.3.2 Ý nghĩa và ứng dụng

Hệ quả trực tiếp của Định lý 5.1 là việc áp dụng hàm làm tròn (rounding) sẽ cho kết quả chính xác tuyệt đối:

$$y_i = \text{sign}(\mu(\hat{x})_i) = \begin{cases} +1 & \text{nếu } \mu(\hat{x})_i > 0 \\ -1 & \text{nếu } \mu(\hat{x})_i \leq 0 \end{cases} \quad (3.6)$$

Điều này biến bộ dự đoán NTK (về bản chất là một bộ xấp xỉ liên tục) thành một hệ thống thực thi chính xác các phép toán rời rạc. Đây là cầu nối quan trọng giữa lý thuyết xấp xỉ hàm của mạng nơ-ron và yêu cầu chính xác tuyệt đối của các thuật toán.

3.4 Kiểm soát tương quan không mong muốn

Để Định lý 5.1 được thỏa mãn, cần kiểm soát chặt chẽ các tương quan không mong muốn giữa các mẫu huấn luyện.

3.4.1 Chi tiết Assumption 5.1

Assumption 5.1: Xét đầu ra của bộ dự đoán NTK cho bit thứ i . Phân tích đầu ra thành hai thành phần:

$$\mu(\hat{x})_i = w^1 \cdot s_i + w^0 \cdot C_i \quad (3.7)$$

trong đó:

- w^1 là **trọng số tín hiệu** (signal weight): đóng góp từ template khớp chính xác
- w^0 là **trọng số nhiễu** (interference weight): đóng góp từ mỗi template không khớp
- $s_i \in \{+1, -1\}$ là giá trị đúng của bit đầu ra
- C_i là **số lượng tương quan không mong muốn** (unwanted correlations)

Một tương quan được gọi là “không mong muốn” khi một template trong tập huấn luyện có:

1. Tích vô hướng khác không với đầu vào $\hat{x}$ (tức là có sự trùng lặp về bit)
2. Nhưng không phải là template đúng cho quy tắc cần áp dụng
3. Và có đầu ra cùng dấu với template đúng (làm tăng nhiễu thay vì triệt tiêu)

3.4.2 Điều kiện đủ để học chính xác

Để đầu ra $\mu(\hat{x})_i$ có dấu đúng, thành phần tín hiệu phải lấn át thành phần nhiễu:

$$|w^1 \cdot s_i| > |w^0 \cdot C_i| \quad (3.8)$$

Vì $|s_i| = 1$, điều kiện này tương đương với:

$$C_i < \frac{|w^1|}{|w^0|} \quad (3.9)$$

Tỷ lệ $|w^1|/|w^0|$ được gọi là *tỷ lệ quyết định* (decision ratio). Assumption 5.1 yêu cầu số lượng tương quan không mong muốn phải nhỏ hơn tỷ lệ này.

3.4.3 Xác minh cho các thuật toán cụ thể

Nghiên cứu chứng minh rằng các thuật toán được xem xét đều thỏa mãn Assumption 5.1:

- **Phép hoán vị:** $C_i = 0$ (không có tương quan không mong muốn do các vị trí bit độc lập)
- **Phép cộng:** $C_i \leq 1$ (tối đa 1 tương quan từ cấu hình bit nhớ)

- **Phép nhân:** $C_i = O(\log l)$ (tăng chậm theo số bit)
- **SBN:** $C_i = O(\log l)$ (tương tự phép nhân)

Trong khi đó, tỷ lệ quyết định $|w^1|/|w^0|$ tăng theo $\Omega(l)$ với l là số bit. Do đó, điều kiện $C_i < |w^1|/|w^0|$ được thỏa mãn khi l đủ lớn.

3.5 Triển khai thuật toán cụ thể

Phần này mô tả cách áp dụng khung template matching cho bốn thuật toán cụ thể.

3.5.1 Phép hoán vị

Phép hoán vị $\pi : [l] \rightarrow [l]$ ánh xạ mỗi vị trí bit sang vị trí mới. Đây là thuật toán đơn giản nhất trong khung template matching.

Mã hóa template: Với mỗi vị trí đầu ra $i \in [l]$, tạo một template đơn giản:

- Đầu vào: Vector có 1 tại vị trí $\pi^{-1}(i)$, 0 tại các vị trí khác
- Đầu ra: Giá trị bit tại vị trí i

Số lượng template: Tổng cộng l template, mỗi template cho một vị trí đầu ra.

Tính chất: Do các template hoàn toàn trực giao (mỗi template chỉ “nhìn” một vị trí đầu vào duy nhất), không có tương quan không mong muốn ($C_i = 0$), và phép hoán vị được học chính xác với độ phức tạp tối thiểu.

3.5.2 Phép cộng

Phép cộng hai số nhị phân l -bit được triển khai thông qua mô phỏng bộ cộng ripple-carry. Thuật toán hoạt động qua hai giai đoạn:

Giai đoạn 1 - Cộng bitwise: Tính tổng XOR và bit nhớ ban đầu cho mỗi cặp bit:

$$s_i^{(0)} = a_i \oplus b_i \quad (\text{tổng tạm thời}) \quad (3.10)$$

$$c_i^{(0)} = a_i \wedge b_i \quad (\text{bit nhớ sinh ra}) \quad (3.11)$$

Giai đoạn 2 - Lan truyền bit nhớ: Cập nhật tuần tự từ bit thấp đến bit cao:

$$s_i^{(t+1)} = s_i^{(t)} \oplus c_{i-1}^{(t)} \quad (3.12)$$

$$c_i^{(t+1)} = (s_i^{(t)} \wedge c_{i-1}^{(t)}) \vee c_i^{(t)} \quad (3.13)$$

Sau l bước lan truyền, $s^{(l)}$ chứa kết quả cộng chính xác.

Số lượng template: $O(l)$ template cho mỗi giai đoạn, tổng cộng $O(l)$.

Tương quan không mong muốn: Tối đa 1 tương quan cho mỗi bit đầu ra, phát sinh từ cấu hình bit nhớ đặc biệt.

3.5.3 Phép nhân

Phép nhân hai số nhị phân l -bit được triển khai bằng thuật toán shift-and-add cổ điển.

Thuật toán: Duy trì ba thanh ghi: tích lũy (accumulator) A , số nhân (multiplier) M , và số bị nhân (multiplicand) B . Tại mỗi bước:

1. **Kiểm tra LSB:** Nếu bit thấp nhất của M là 1, cộng B vào A
2. **Dịch phải:** Dịch cặp (A, M) sang phải 1 bit
3. **Lặp:** Tiếp tục cho đến khi xử lý hết l bit của M

Mã hóa template: Mỗi bước của thuật toán được mã hóa thành các template riêng biệt cho:

- Quy tắc cộng có điều kiện (phụ thuộc LSB)
- Quy tắc dịch bit
- Quy tắc cập nhật thanh ghi

Số bước: $O(l)$ bước lặp, mỗi bước có $O(l)$ template.

Tương quan không mong muốn: $O(\log l)$ do sự tương tác giữa các bit trong phép cộng và dịch.

3.5.4 Chỉ thị SBN (Subtract and Branch if Negative)

Chỉ thị SBN là một chỉ thị đơn giản nhưng có sức mạnh biểu diễn Turing-complete. Mỗi chỉ thị SBN có dạng (a, b, c) và thực hiện:

1. Tính $\text{Mem}[a] := \text{Mem}[a] - \text{Mem}[b]$
2. Nếu kết quả âm, nhảy đến địa chỉ c ; ngược lại, tiếp tục tuần tự

Tính Turing-complete: Đã được chứng minh rằng mọi hàm tính toán được (computable function) đều có thể được biểu diễn bằng một chương trình chỉ sử dụng chỉ thị SBN. Do đó, nếu mạng nơ-ron có thể học thực thi chính xác SBN, về nguyên tắc nó có khả năng tính toán bất kỳ hàm tính toán được nào.

Triển khai: SBN được phân tách thành các thành phần:

- **Phép trừ:** Giảm về phép cộng với số bù 2 (two's complement)
- **Kiểm tra dấu:** Kiểm tra bit cao nhất của kết quả
- **Rẽ nhánh:** Cập nhật thanh ghi địa chỉ dựa trên kết quả kiểm tra

Ý nghĩa: Kết quả về SBN khẳng định tính phổ quát của khung lý thuyết: mạng nơ-ron không chỉ học được các phép toán số học cơ bản mà còn có khả năng, về nguyên tắc, học thực thi bất kỳ thuật toán nào.

3.6 Độ phức tạp tập hợp

Trong thực tế, các mạng nơ-ron hữu hạn có phương sai đáng kể xung quanh giá trị kỳ vọng lý thuyết. Để đảm bảo độ chính xác cao, nghiên cứu sử dụng phương pháp ensemble.

3.6.1 Tại sao cần Ensemble

Bộ dự đoán NTK trong giới hạn infinite-width cho kết quả chính xác về kỳ vọng $\mu(\hat{x})$. Tuy nhiên, với mạng hữu hạn chiều rộng m , đầu ra thực tế là:

$$\hat{y} = \mu(\hat{x}) + \epsilon \quad (3.14)$$

trong đó ϵ là nhiễu ngẫu nhiên với phương sai σ^2 phụ thuộc vào m và cấu trúc bài toán.

Nếu $|\mu(\hat{x})_i|$ nhỏ so với σ , nhiễu có thể đảo dấu của đầu ra, dẫn đến kết quả sai. Ensemble giải quyết vấn đề này bằng cách lấy trung bình đầu ra của nhiều mô hình:

$$\bar{y} = \frac{1}{N} \sum_{j=1}^N \hat{y}^{(j)} \quad (3.15)$$

Theo định luật số lớn, phương sai của trung bình giảm theo σ^2/N , cho phép đạt độ chính xác mong muốn với N đủ lớn.

3.6.2 Công thức tính kích thước Ensemble

Để đảm bảo xác suất lỗi không vượt quá δ , số lượng mô hình trong ensemble cần thỏa mãn:

$$N \geq \frac{2\sigma^2}{\mu_{\min}^2} \ln \left(\frac{2}{\delta} \right) \quad (3.16)$$

trong đó:

- σ^2 là phương sai của một mô hình đơn lẻ
- $\mu_{\min} = \min_i |\mu(\hat{x})_i|$ là giá trị kỳ vọng nhỏ nhất (theo giá trị tuyệt đối)
- δ là xác suất lỗi cho phép

Công thức này được suy ra từ bất đẳng thức Hoeffding và đảm bảo rằng với xác suất ít nhất $1 - \delta$, trung bình ensemble có dấu đúng cho tất cả các bit đầu ra.

3.6.3 Độ phức tạp theo số bit

Phân tích chi tiết cho thấy với các thuật toán được xem xét:

- $\sigma^2 = O(l)$ (phương sai tăng tuyến tính theo số bit)
- $\mu_{\min}^2 = \Omega(1/l)$ (kỳ vọng nhỏ nhất giảm theo $1/\sqrt{l}$)

Do đó, kích thước ensemble cần thiết là:

$$N = O(l^2 \cdot \log(1/\delta)) \quad (3.17)$$

Kết quả này có ý nghĩa thực tiễn quan trọng: độ phức tạp ensemble tăng theo $O(l^2)$, tức là đa thức bậc hai theo số bit. Điều này cho thấy phương pháp có khả năng mở rộng (scalable) cho các bài toán lớn, mặc dù chi phí tính toán tăng khi kích thước đầu vào tăng.

3.6.4 So sánh với độ phức tạp mẫu

Một điểm đáng chú ý là trong khi độ phức tạp ensemble là $O(l^2)$, độ phức tạp mẫu (số lượng ví dụ huấn luyện cần thiết) chỉ là $O(\log l)$ cho các thuật toán cơ bản. Sự kết hợp giữa hiệu quả dữ liệu (logarithmic sample complexity) và khả năng mở rộng ensemble (polynomial ensemble complexity) làm cho phương pháp này trở nên hấp dẫn cho các ứng dụng thực tế đòi hỏi độ chính xác tuyệt đối.

Chương 4

THỰC NGHIỆM VÀ PHÂN TÍCH

4.1 Thiết lập thực nghiệm

4.2 Xác minh lý thuyết

4.2.1 Kết quả trên phép cộng và nhân

4.2.2 So sánh phương sai và kỳ vọng

4.3 Thực nghiệm huấn luyện

4.3.1 Bài toán Hoán vị

4.3.2 Độ chính xác theo kích thước Ensemble

Chương 5

KẾT LUẬN

Tài liệu tham khảo

- [1] Artur Back de Luca, George Giapitzakis, and Kimon Fountoulakis, "Learning to Add, Multiply, and Execute Algorithmic Instructions Exactly with Neural Networks", arXiv:2502.16763, 2025.

Phụ lục A

Chứng minh toán học bổ trợ

Phụ lục B

Mã nguồn cài đặt mô phỏng